

Env-Build-up

- 1.Codebase build up
2. API build up

Baselines

Paper	Method	Metric
Swe-Rebench(NIPS25)	1.LLM scanning all the building related files 2.LLM produce three jsons 3.validate and recycle	31% successful
GSO	No details	No details

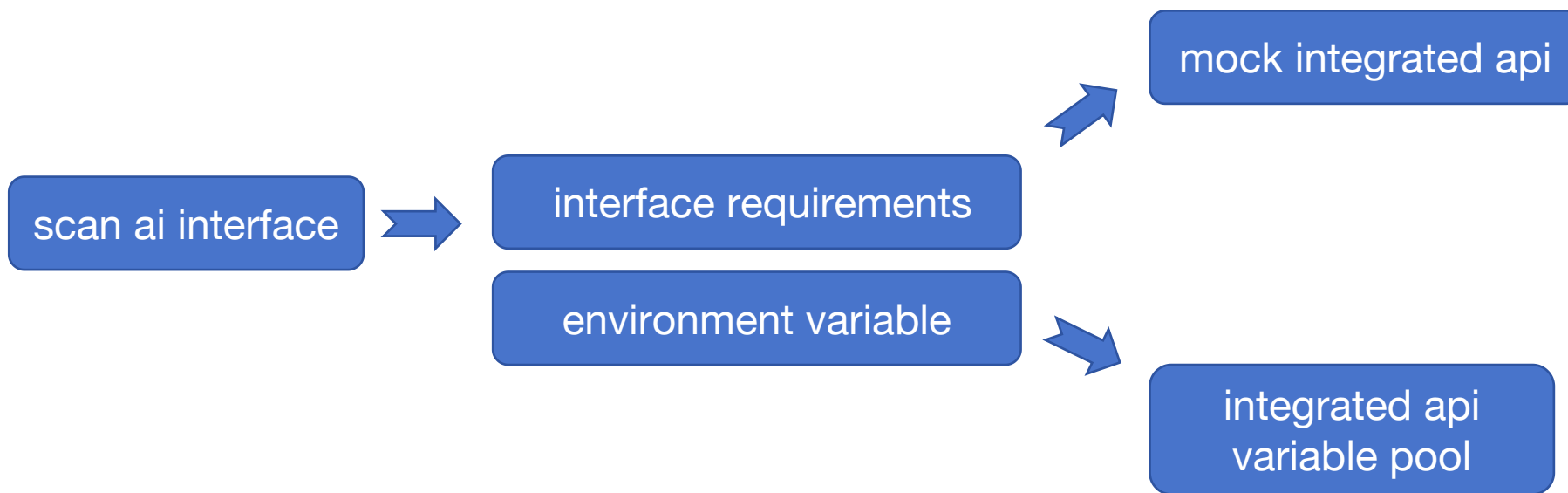
```
def model_post_init(self, __context) -> None:
    self.setup_commands = [
        "sudo apt update -y && sudo apt upgrade -y",
        "sudo apt-get install -y libtiff5-dev libjpeg8-dev libopenjp2-7-dev",
        "zlib1g-dev",
        "sudo apt-get install -y libfreetype6-dev liblcms2-dev libwebp-dev tcl8.6-dev",
        "tk8.6-dev python3-tk",
        "sudo apt-get install -y libharfbuzz-dev libfribidi-dev libxcb1-dev libx11-dev",
        "libssl-dev",
        "sudo apt install -y gcc g++ gfortran libopenblas-dev liblapack-dev",
        "pkg-config",
        "sudo apt-get install -y clang",
        "sudo apt-get install -y llvm",
        "sudo mkdir -p /tools/llvm/bin",
        "sudo ln -s $(which llvm-ar) /tools/llvm/bin/llvm-ar",
    ]

    if self.install_commands == []:
        self.install_commands = [
            f"uv venv --python {self.py_version}",
            "source .venv/bin/activate",
            "which python",
            "python --version",
            "uv pip install -e .",
            "uv pip install requests",
            f"uv pip show {self.repo.repo_name}",
        ]
```

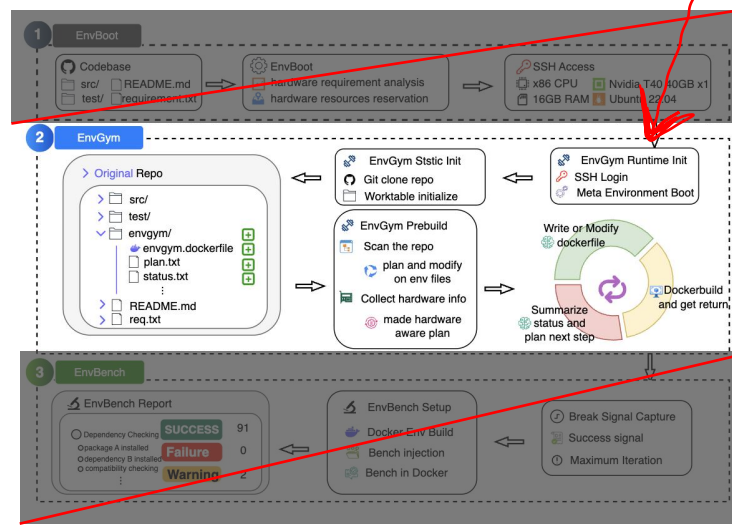
Baselines

Paper	Method	Metric
Sec-Bench	OSS-Fuzz template +SecBench test	retain 898(81%) instances as candidates.
Swe-Smith	Swe-Agent	No details filter only 80% test pass
EnvAgent	1.Scan Repo 2.Check hardware 3.Make plans 4.Step by step build and eval	49/54

api build up



codebase build up



crewAI

```
data > crewAI > ENVIRONMENT_SETUP_PLAN.md
9
- internet connection: REQUIRED for API calls to LLM providers
10
- Project Directory: /home/cc/EnvGym/data/crewAI
11
- Python Version: >=3.10 and <3.14 (3.10 or 3.11 recommended)
12
---
13
14
15 **1. DOWNLOADS NEEDED:**
16 _(Adjusted for multi-platform, API-based LLM usage, no local GPU required)_
17
18 **Python**:
19 - Version: 3.10, 3.11, 3.12, or 3.13 (3.10 or 3.11 recommended for best compatibility)
20   - _Do NOT use 3.9 or below, or 3.14+_
21   - _If using Docker, use 'python:3.10-slim' or 'python:3.11-slim' as base image_
22
23 **pip**: Latest version recommended (pip >= 21.0)
24
25 **UV Package Manager** (RECOMMENDED): Latest version from astral.sh
26   - Alternative: Use pip if UV not available
27
28 **Git**: For cloning repositories
29
30 **CUDA Toolkit**: _SKIP - NOT NEEDED_ (all LLM inference is remote via API)
31
32 **Python Dependencies** (from 'pyproject.toml' in lib/crewai/):
33   - crewai (core library)
34   - crewai-tools (optional, for additional agent tools)
35   - openai >= 1.13.3 (for OpenAI provider)
36   - anthropic (for Anthropic/Claude provider)
37   - google-generativeai (for Google Gemini provider)
38   - boto3 (for AWS Bedrock provider)
39   - Other provider SDKs as needed
40
41 **Required API Keys** (at least ONE):
42   - OpenAI API Key (from platform.openai.com)
43   - Anthropic API Key (from console.anthropic.com)
44   - Google Gemini API Key (from ai.google.dev)
45   - Azure OpenAI credentials (from Azure Portal)
46   - AWS Bedrock credentials (AWS Access Key, Secret Key, Region)
47   - Groq API Key (from console.groq.com)
48   - Or Ollama (local, self-hosted models)
49
50 **Optional API Keys**:
51   - Serper API Key (for web search tool)
52   - AgentOps API Key (for observability)
53
54 **Development Tools** (optional):
55   - pre-commit (for code quality checks)
```

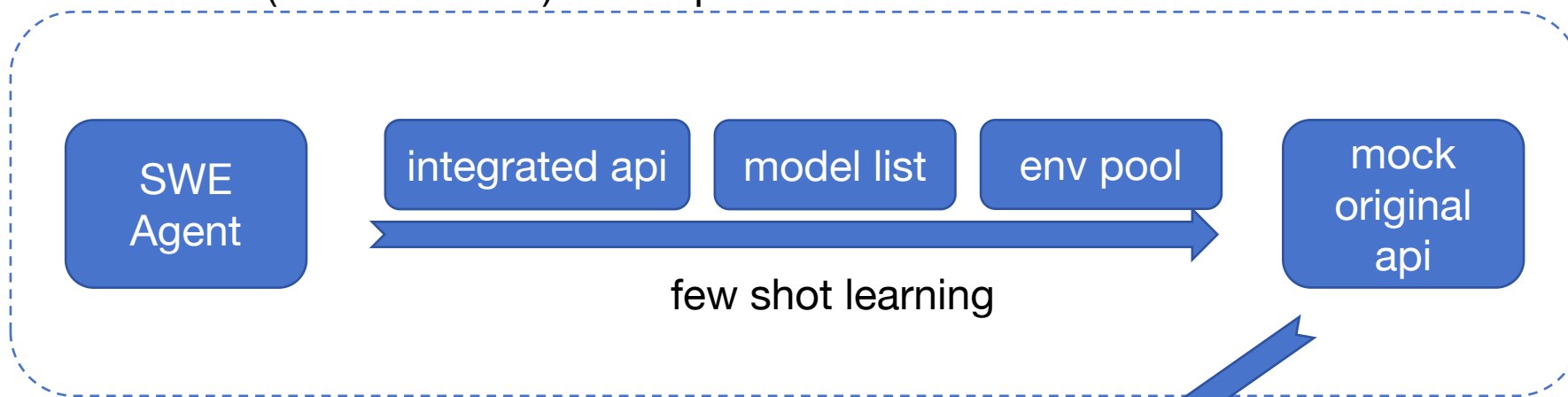
```
56
57 7. **Obtain API Keys** (at least ONE required):
58   - Anthropic: Visit console.anthropic.com
59   - Google Gemini: Visit ai.google.dev/apikey
60   - Azure OpenAI: Configure in Azure Portal
61   - AWS Bedrock: Configure AWS credentials
62   - Groq: Visit console.groq.com
63   - Serper (optional): Visit serper.dev
64
65 8. **Create a new CrewAI project**:
66   - `crewai create crew your-project-name`
67   - `cd your-project-name`
68   - This generates project structure with .env, agents.yaml, tasks.yaml, etc.
69
70 9. **Configure .env file** (CRITICAL):
71   - Edit `.env` in project root
72   - Add at least ONE LLM provider API key:
73     ```bash
74     # Choose at least ONE:
75     OPENAI_API_KEY=sk-proj-...
76     # ANTHROPIC_API_KEY=sk-ant-...
77     # GEMINI_API_KEY=AIza...
78     # AZURE_API_KEY=...
79     # AZURE_ENDPOINT=https://your-resource.openai.azure.com
80
81     # Optional tools:
82     # SERPER_API_KEY=...
83
84     # Optional model selection:
85     # MODEL=gpt-4o-mini
86     ```
87   - **Never commit .env to git**
88   - Ensure `.env` is in `.gitignore`
89
90 10. **Update .gitignore**:
91   - Ensure .env is excluded
92   - Ensure venv/, __pycache__/, .DS_Store, etc. are excluded
93
94 11. **Install project dependencies**:
```

problem 1.cost (32min with \$2.31)

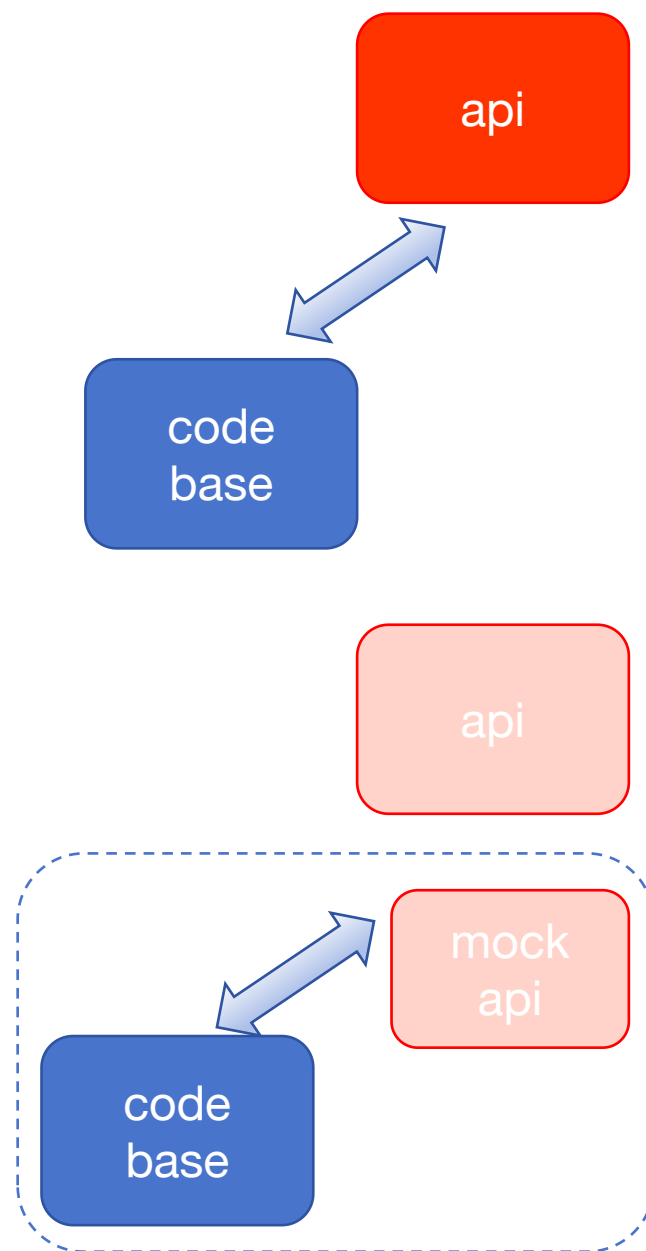
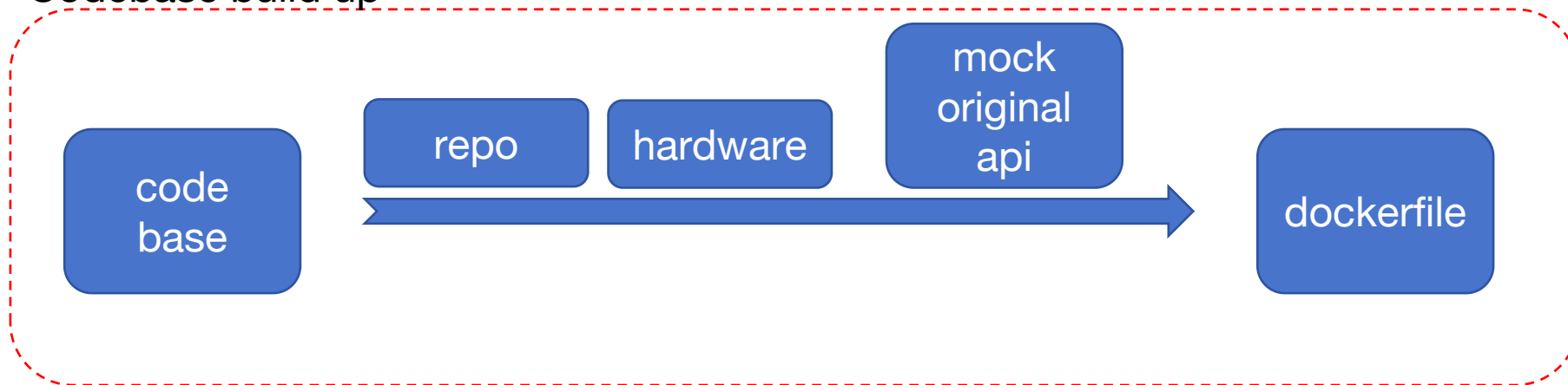
problem 2.need manual test on built dockerfile(for now i only know the dockerfile is runnable)

more easy way:

AI resources(API and so on) build up



Codebase build up



api build up

env analysis and
config in dockerfile: ☒

mock in unittest (build in test)

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim

# Install uv globally
RUN pip install uv

# Set workdir to repo root
WORKDIR /home/cc/SWEGENT-BENCH/crewAI

# Copy repo contents into Docker image
COPY . .

# Install all dependencies as per pyproject.toml and uv.lock
RUN uv sync

# Forge/OpenAI/Anthropic API env vars
ENV OPENAI_BASE_URL="https://api.forge.tensorblock.co/v1"
ENV OPENAI_API_KEY="${FORGE_API_KEY}"
ENV ANTHROPIC_BASE_URL="https://api.forge.tensorblock.co"
ENV ANTHROPIC_AUTH_TOKEN="${FORGE_API_KEY}"
ENV FORGE_API_KEY="your-forge-api-key-here"
ENV FORGE_BASE_URL="https://api.forge.tensorblock.co/v1"
ENV MODEL="OpenAI/gpt-4.1"
ENV AI_TEMPERATURE=0.7
ENV AI_MAX_TOKENS=1000
ENV AI_TOP_P=1
ENV AI_FREQUENCY_PENALTY=0
ENV AI_PRESENCE_PENALTY=0

# Default to bash shell at repo root
CMD ["/bin/bash"]
```

mock original OpenAI api or other api

```
class TestAIClient(unittest.TestCase):
    @patch("my_ai_client.OpenAI")
    def test_get_chat_response(self, MockOpenAI):
        # Create a mock return object
        mock_completion = MagicMock()
```